

DEV2 – JAVL – Laboratoire Java

TD 03 – Listes et boucles for

Surcharge, exception, liste, boucle for et foreach

Résumé

Au terme de ce TD vous serez capable de :

- ▷ répéter une ou plusieurs instructions un certain nombre de fois (boucle for)
- ▷ identifier et mettre en œuvre la surcharge d'une méthode
- ▷ lever (lancer, déclencher) une exception
- ▷ choisir l'exception appropriée entre `IllegalStateException` et `IllegalArgumentException`
- ▷ parcourir une liste en parcourant les valeurs (boucle foreach)

Table des matières

1 Aboyer plusieurs fois	1
1.1 Répéter : boucle for	1
1.2 Surcharge de méthode	3
1.3 Conclusion	3
2 Lever des exceptions	4
3 Annuaire (listes et foreach)	5

1 Aboyer plusieurs fois

Aux séances précédentes, vous avez créé un programme définissant le concept de chien. Un chien possède un nom et un poids, et peut avoir faim. Un chien peut aboyer, courir, manger.

1.1 Répéter : boucle for

Pour faire aboyer un chien plusieurs fois, nous pouvons imaginer répéter les appels à la méthode `bark()`.

Nous pouvons imaginer faire plusieurs appels :

```
public class Main {  
    public static void main(String[] args) {  
        Dog medor = new Dog("Médor");  
        medor.bark();  
        medor.bark();  
        medor.bark();  
    }  
}
```

ou, de préférence, avec une boucle *for* :

```
public class Main {  
    public static void main(String[] args) {  
        Dog medor = new Dog("Médor", 20);  
        for (int i = 0; i < 3; i++) {  
            medor.bark();  
        }  
    }  
}
```

Si vous n'avez jamais utilisé cette syntaxe, c'est le moment de vous l'approprier : pendant quelques minutes, utilisez cette syntaxe de boucle en modifiant le 0, le 3, ou encore le nom de la variable *i*. Notez l'utilisation des accolades contrairement à un langage comme Python.

Boucle for en Java En Java, la boucle *for* existe en deux variantes. La variante historique ressemble à ceci :

```
for (initialisation; condition; incrémentation) {  
    instructions...  
}
```

(Notez les parenthèses, les point-virgules et les accolades.) L'ordre d'exécution des quatre éléments est :

1. initialisation
2. condition : si la condition s'évalue à **false**, tout s'arrête et le programme continue après la boucle
3. instructions
4. incrémentation
5. (retour à l'étape 2 : la condition)

La variante plus récente, appelée *foreach* ou *enhanced for*, ressemble à ceci :

```
for (Type variable : objetItérable) {  
    instructions...  
}
```

(Notez les parenthèses, la déclaration de type, et les deux-points.) Ceci correspond à peu près au code python

```
for variable in objetItérable:  
    instructions...
```

Exercice 1

La boucle for

Quelques petites demandes fort simples pour manipuler ces boucles *for* :

- ☒ Faites aboyer Médor 5 fois plutôt que 3.
- ☒ Ajoutez (si vous l'aviez enlevé) un chien nommé Milou, et faites-le aboyer 5 fois également. Il y a deux variantes :
 - ▷ faire aboyer cinq fois Médor puis cinq fois Milou, ou
 - ▷ faire aboyer cinq fois en alternance Médor et Milou (l'un, puis l'autre, puis l'un, puis l'autre, etc.)

Essayez les deux !

Astuce

IntelliJ propose un raccourci pour ré-indenter (ou re-formatter) votre code ^a. Utilisez-le souvent !

^a. IntelliJ propose aussi un menu permettant d'accéder à une liste de raccourcis (shortcuts). Trouvez cette liste, puis trouvez le raccourci pour ré-indenter.

1.2 Surcharge de méthode

L'objectif de cette section est de mettre en œuvre et comprendre la surcharge de méthode. Nous commençons par un premier exercice ne faisant pas intervenir de surcharge.

Exercice 2

Ajouter une méthode

Ajoutez à présent une méthode `barkMultiple` (dans la classe `Dog`) qui permet de faire aboyer un chien plusieurs fois. Vous avez normalement tous les ingrédients pour y parvenir : faites-le ! Voici les étapes principales :

- ✍ Ajouter une méthode nommée `barkMultiple` acceptant un paramètre `int nbRepeat`. (Pour vous aider, re-voyez comment vous avez codé `Contact::setPhone`.)
- ✍ Dans le corps de la méthode, utilisez une boucle `for` qui fait appel à la méthode `bark`.
- ✍ Testez cette nouvelle méthode en appelant `medor.barkMultiple(7)` depuis votre programme principal.

Exercice 3

Surcharger la méthode existante

Une fois l'exercice terminé et fonctionnel, changez le nom de la méthode `barkMultiple` : appelez la simplement `bark`. Notez que c'est le même nom que la méthode `bark` initiale mais que Java ne se plaint pas : Java les différencie parce que l'une accepte un paramètre entier, et l'autre n'a aucun paramètre. On dit donc que c'est la *signature* qui est différente.

Quand plusieurs méthodes d'une même classe ont le même nom (et des signatures différentes), on parle de *surcharge de méthode*.

1.3 Conclusion

Voici à quoi devrait ressembler le code de vos méthodes `bark` :

```
void bark() {
    System.out.println(this.name + " a fait: Wouff!");
}

void bark(int nbRepeat) {
    for (int i = 0; i < nbRepeat; i++) {
        this.bark();
    }
}
```

2 Lever des exceptions

Actuellement, votre méthode `run` ressemble sans doute à ceci :

```
void run() {
    if (this.hungry) {
        System.out.println(this.name + " a faim !");
    } else {
        System.out.println(this.name + " court comme un fou !");
        this.hungry = true;
    }
}
```

En particulier cette méthode ne fait pas toujours courir le chien. Il arrive que le chien ne coure pas, et cela passe finalement relativement inaperçu (il n'y a qu'un message indiquant qu'il a faim).

Pour pallier à cela, modifiez la méthode `run` pour *lever une exception* lorsque le chien a faim :

```
void run() {
    if (this.hungry) {
        throw new IllegalStateException(this.name + " a faim !");
    }
    System.out.println(this.name + " court comme un fou !");
    this.hungry = true;
}
```

Maintenant faites en sorte d'appeler deux fois la méthode `run()` pour constater ce qui se passe.

Si tout va bien, votre programme s'arrête abruptement (il plante, il *crash*). Nous avons donc mis en place un procédé qui fait planter votre programme dans certaines circonstances. L'avantage est qu'en cas de problème, nous sommes très rapidement au courant (un programme qui plante, ça se remarque mieux qu'un programme qui affiche simplement un message sur la console). Le désavantage est qu'un programme qui plante, ce n'est jamais très agréable. Nous finirons par apprendre à gérer cela avec des `try-catch`.

Exercice 4 Ne pas manger trop

Ajustez la méthode `eat` afin de lever une exception lorsqu'elle est appelée et que le chien n'a pas faim.

Exercice 5 Ne pas aboyer à l'envers

Ajustez la méthode `bark(int nbRepeat)` afin de lever une exception lorsqu'elle est appelée avec un paramètre négatif (ou nul). Attention : cette fois l'exception à lever sera `IllegalArgumentException`, puisque `nbRepeat` est un argument de la méthode plutôt qu'un état du chien.

Exercice 6 En cas de blessure

À vous de jouer :

- ✍ Ajoutez un attribut booléen `injured` indiquant si le chien est blessé ou pas. Dans le constructeur¹, assurez vous que le chien n'est pas blessé lorsqu'il est instancié.
- ✍ Ajustez la méthode `run` pour qu'un chien blessé ne puisse pas courir.

1. Nous mentionnons ici le constructeur mais vous pouvez aussi donner la valeur `false` à la déclaration de l'attribut. C'est bien de garder une certaine cohérence. En réalité, la valeur `false` est déjà la valeur par défaut d'un attribut ; vous pourriez donc omettre entièrement de la définir.

- ✎ Ajustez encore la méthode `run` pour qu'un chien qui court puisse se blesser avec une probabilité d'une chance sur cinq².
- ✎ Ajustez enfin la méthode `eat` pour qu'un chien qui mange se rétablisse complètement, et ne soit donc plus blessé.

3 Annuaire (listes et foreach)

Aux séances précédentes, vous avez créé une classe définissant le concept de `Contact`. Un contact possède un prénom, un nom, une adresse et un numéro de téléphone.

Vous avez peut-être écrit du code tel que celui-ci :

```
Contact moi = new Contact("Albert", "Einstein", "rue d'Ailleurs", 12345);
Contact unAutre = new Contact("Mahatma", "Gandhi", "rue d'Autre Part", 54321);
```

Exercice 7

Une classe Annuaire

- ✎ Créez une classe `Phonebook`.
- ✎ Ajoutez-lui un attribut `contacts` (au pluriel!) de type `List<Contact>` (vous devrez faire un `import`. Demandez à IntelliJ de vous aider, et lisez l'encadré ci-dessous.).
- ✎ Ajoutez un constructeur sans paramètre. Dans le constructeur vous définirez votre liste de contacts : ce sera une liste vide. `this.contacts = new ArrayList<>();`
- ✎ Ajoutez une méthode `add(Contact c)` qui permet d'ajouter un contact existant. Notez que vous pouvez utiliser `this.contacts.add(...)` pour ajouter quelque chose dans la liste des contacts.
- ✎ Ajoutez une méthode `display()` qui permet d'afficher le contenu de l'annuaire. Voici un embryon de code.

```
for (Contact c : this.contacts) {
    System.out.println(c.getFullname());
}
```

À vous de le compléter pour afficher aussi le numéro de téléphone et l'adresse.

Notion de liste

Le type `List<Contact>` est un type adapté pour une liste de contacts. Bien qu'en Python une liste peut contenir n'importe quoi, en Java il faut préciser le type de ce qui se trouve dans la liste. Nous indiquons donc que c'est une liste, mais plus précisément que c'est une liste d'objets de type `Contact`.

2. Ici, une petite recherche sera nécessaire. Vous aurez peut-être besoin de faire appel à une classe externe, fournie par Java, similairement à ce que vous avez déjà pu faire en Python. Si un `import` est nécessaire, votre IDE (IntelliJ) pourra le faire pour vous.

List et API Java

Le type `List<...>` fait partie de l'API Java : il existe et est disponible, mais il faut une instruction *import* pour pouvoir l'utiliser.

Plus précisément, le nom complet de ce type est `java.util.List` : c'est la classe `List` du *package* `java.util`. Pour pouvoir écrire seulement `List`, on doit ajouter en début de fichier l'instruction `import java.util.List`.

Encore deux remarques :

- ▷ Chaque classe définit un type. L'aviez-vous remarqué ? `List`, `Contact`, et désormais également `Phonebook`.
- ▷ Il arrivera encore que vous deviez ajouter un *import*. Votre IDE (IntelliJ) vous le rappellera généralement (ou parfois le fera pour vous). Vous l'aviez peut-être déjà constaté avec `ArrayList` ?

Interface et implémentation

L'attribut `contacts` est initialisé avec une liste vide mais, plutôt que d'écrire `new List()`, nous écrivons `new ArrayList<>()`. Nous vivons avec cette étrangeté pour l'instant. Nous y reviendrons lorsque nous parlerons d'interface et d'implémentation.

Exercice 8

Utilisation de la classe `Annuaire`

Écrivez le code suivant dans votre méthode `main` :

```
Contact moi = new Contact("Albert", "Einstein", "rue d'Ailleurs", 12345);
Contact unAutre = new Contact("Mahatma", "Gandhi", "rue d'Autre Part", 54321);
Phonebook annuaire = new Phonebook();
annuaire.add(moi);
annuaire.add(unAutre);
annuaire.add(new Contact("Nelson", "Mendela", "Avenue l'eau de là", 123321));
annuaire.display();
```

et vérifiez que tout fonctionne.